

Хук `useLayoutEffect`



Статус

Готово

Хук `useLayoutEffect`

У цьому розділі розглянемо рідко використовувані або нішеві React-хуки. Вони не є частиною обов'язкової бази, а скоріше додатковим матеріалом для розширення кругозору. Достатньо знати про їхнє існування та базове призначення. Глибоко занурюватися не обов'язково — повернетеся до них за потреби.

У React, окрім стандартного `useEffect`, існує ще хук `useLayoutEffect`. Цей хук працює майже так само, як `useEffect`, але з однією важливою відмінністю:

- `useEffect` запускається **після того**, як браузер намалював зміни на екрані (після етапу `paint`).
- `useLayoutEffect` запускається **до того**, як браузер виконає малювання (до `paint`).

Тобто `useLayoutEffect` виконується синхронно одразу після оновлення DOM, але ще до того, як браузер покаже зміни користувачу. Це дозволяє уникнути небажаного «миготіння» інтерфейсу.

Для чого потрібен `useLayoutEffect` ?

`useLayoutEffect` зазвичай використовується коли потрібно:

- виміряти розміри елементів після оновлення DOM (наприклад, через `getBoundingClientRect`);
- синхронно змінити стилі елементів одразу після оновлення;
- виконати позиціонування або анімацію без помітного стрибка чи мерехтіння.

`useEffect` для таких завдань не підходить, бо він спрацьовує вже після того, як браузер намалював новий DOM, і користувач може встигнути помітити проміжний стан.

Алгоритм роботи

1. React оновлює DOM.
2. Викликається `useLayoutEffect` (синхронно).
3. Виконуються вимірювання або зміни DOM.
4. Тільки потім браузер малює оновлений інтерфейс (paint).
5. Після цього викликається `useEffect`.

Приклади

У цьому прикладі ми вимірюємо ширину кнопки після рендеру:

```
import { useLayoutEffect, useRef, useState } from 'react';

export default function App() {
  const buttonRef = useRef<HTMLButtonElement | null>(null);
  const [width, setWidth] = useState(0);

  useLayoutEffect(() => {
    if (buttonRef.current) {
      const rect = buttonRef.current.getBoundingClientRect();
      setWidth(rect.width);
    }
  }, [buttonRef]);
}
```

```

    }
    }, []);

    return (
      <>
        <button ref={buttonRef}>Click me</button>
        <p>Button width: {width}px</p>
      </>
    );
  }
}

```

Якби ми використали `useEffect`, значення ширини оновилося б після малювання, і користувач міг би помітити «стрибок» тексту `0` до поточного значення.

Якщо ви не бачите різниці між `useEffect` та `useLayoutEffect` у цих прикладах — це абсолютно нормально. У сучасних браузерах і на швидких комп'ютерах оновлення DOM настільки швидке, що миготіння майже непомітне.

Проте різниця принципова:

- `useLayoutEffect` гарантовано виконується одразу після рендеру, але **до** того, як браузер покаже зміни користувачу. Це дає змогу синхронно працювати з DOM без ризику проміжного некоректного стану.
- `useEffect` спрацьовує вже **після** того, як браузер відмалює сторінку. Теоретично між рендером і ефектом може пройти коротка затримка, і на слабших пристроях або повільних мережах користувач може помітити стрибок.

Якщо у вас зараз різниці не видно — це добре, але варто розуміти, що для надійної синхронної взаємодії з DOM краще використовувати `useLayoutEffect`.

Ще один класичний приклад — потрібно синхронно перемістити скрол позицію.

```

import { useEffect, useRef } from 'react';

export default function App() {
  const containerRef = useRef<HTMLDivElement | null>(null);

  useEffect(() => {
    if (containerRef.current) {
      containerRef.current.scrollTo(0, containerRef.current.scrollHeight);
    }
  }, []);

  return (
    <div
      ref={containerRef}
      style={{
        overflowY: 'auto',
        height: 200,
        border: '1px solid gray'
      }}
    >
      {Array.from({ length: 20 }, (_, i) => (
        <p key={i}>Line {i + 1}</p>
      ))}
    </div>
  );
}

```

У цьому прикладі використовується `useEffect`, бо він спрацьовує одразу після рендеру, ще до того, як браузер покаже контент користувачу. Завдяки цьому скролл синхронно виставляється в правильну позицію, і користувач не бачить миготіння чи стрибка.

Якщо замість нього використати `useEffect`, скролл оновиться вже після того, як контент з'явиться на екрані, і користувач встигне помітити некоректну позицію під час першого відображення.

Коли краще `useEffect` , а коли `useLayoutEffect` ?

`useEffect` — оптимальний для більшості завдань:

- HTTP-запити
- підписки на події
- робота з `localStorage`
- аналітика

`useLayoutEffect` застосовується, коли:

- треба виміряти DOM
- синхронно змінити DOM перед відображенням
- виконати точне позиціонування або анімацію

Більшість ваших компонентів повинні працювати через `useEffect` . Використовувати `useLayoutEffect` варто лише тоді, коли без нього неможливо досягти потрібного результату.

Підсумок

- `useLayoutEffect` — це «синхронний» ефект, який виконується до етапу відображення (paint).
- `useLayoutEffect` виконується синхронно і може затримати малювання інтерфейсу, тому не слід ним зловживати.
- Підходить для вимірювання елементів або зміни DOM одразу після рендеру.
- Використовується точково, коли `useEffect` не підходить.
- Не варто застосовувати його без потреби, щоб не впливати на швидкодію.
- У середовищі серверного рендерингу (`SSR`) `useLayoutEffect` працює так само, як `useEffect` , адже на сервері немає DOM.